

华睿工业相机 SDK 开发手册

(Python 篇)

Ver1.1

变更履历

编号	变更日期	版本号	变更内容概要	配套 SDK 版本
1	2023/07	V1.0	初版做成	2.3.1
2	2024/03	V1.1	新增多相机配套例程、保存图片接口	2.4.2

目录

1. SDK 概要说明	4
1.1. SDK 支持的操作系统	4
1.2. SDK 安装目录	4
1.2.1. Windows 版 SDK	4
1.2.2. Linux 版 SDK	6
1.3. SDK（Python 接口）对应的资源	7
1.4. SDK 工程开发配置	7
2. SDK 的整体调用流程	8
3. SDK 的各功能接口调用说明	9
3.1. 系统操作	9
3.1.1. 获取版本信息	9
3.1.2. 枚举设备（发现设备）	9
3.1.3. 设备句柄的创建与销毁	10
3.2. 相机操作	10
3.2.1. 相机的连接与关闭	10
3.2.2. 相机配置的下载、保存及加载配置	11
3.2.3. 相机配置设置（GigE 相机特有）	12
3.2.4. 本品牌相机特有功能	13
3.3. 注册事件回调	14
3.4. 相机流资源操作	15
3.4.1. 流数据设置	15
3.4.2. 图像采集	16
3.4.3. 获取图像数据	17
3.4.4. 流与图像数据的其他操作	18
3.5. 属性操作	18
3.5.1. 属性的判断、获取、设置	18
3.5.2. 命令属性的执行	23
3.5.3. 属性的查询与使用	24
3.6. 图像操作	31
3.6.1. 录像相关	31
3.6.2. 图像相关	31
4. 配套例程说明	33
5. 常见问题&注意点	34
5.1. 使用 python 接口采图，只能取到 8 张图像（和 SDK 缓存个数一致）。	34

1. SDK 概要说明

1.1. SDK 支持的操作系统

SDK 支持以下版本的操作系统

操作系统类型	操作系统版本号	备注
Windows *1	Win7 SP1 32/64bit	务必注意带 SP1 补丁
	Win8 32/64bit	
	Win10 32/64bit	
Linux (x86) *2	系统版本： Ubuntu 12.04 LTS ~ Ubuntu 21.04 LTS （32/64bit）	不包括升级了内核版本的系统
	CentOS 6.5 ~ CentOS 8.0 （32/64bit）	不包括升级了内核版本的系统

*1. 从 SDK2.2.5 开始已经不支持 windows xp 系统

*2. 目前支持的 Linux 内核版本号从 2.6.32（含）到 5.11.0（含）。
可以通过 `uname -a` 命令来确定系统内核版本号。

1.2. SDK 安装目录

1.2.1. Windows 版 SDK

Windows 版本 SDK 的安装目录结构如下。

※ 从 Ver2.1.0 版本开始，用户可以自己选择安装位置。

名称	修改日期	类型
 Application	2022/1/17 11:33	文件夹
 Development	2022/1/17 11:33	文件夹
 Documentations	2022/1/17 11:33	文件夹
 Drivers	2022/1/17 11:33	文件夹
 Runtime	2022/1/17 11:33	文件夹

文件夹说明如下：

Application	1.相机客户端软件，包含了 32 位与 64 位 2.相机配套工具软件，包含了 32 位与 64 位
Development	3rdPartyPlatformAdapter 包含了两个第三方算法平台（Halcon、Sherlock）插件动态库
	DOTENT DLL 包含.Net 库文件，在创建项目时从中导入
	Include 包含了 SDK 的接口头文件，头文件中解释了各个接口的基础属性，具体用法在 samples 文件夹中有示例 另一个是 SDK 扩展定义头文件，定义了 SDK 的错误码、参数自定义类型、些许参数的数据限制、及各类行为方式的枚举、定义了多种信息的数据结构
	Lib 包含 lib 库文件
	Samples 包含 demo 示例（C#、Delphi、Lab View、MFC、Python、QT、VB、VC）
Documentations	.Net 接口文档

	ActiveX 接口文档 C 接口文档 SDK 开发手册
Drivers	DShowFilter 的安装与卸载程序 GigE 高性能驱动的安装与卸载程序 PCIex86 智能相机驱动的安装与卸载程序 USB 高性能驱动的安装与卸载程序
Runtime	SDK 运行时编译动态链接库 包含第三方 GenICam 库 分为 32 位与 64 位两个文件夹

1.2.2. Linux 版 SDK

Linux 版本 SDK 的安装目录固定在/opt/公司名/产品名, 文件夹结构、以及概要说明如下:

MVviewer

- └ bin : GUI工具, 发现、连接相机, 拉流取图, 设置相机属性, 卸载脚本, etc
- └ include : SDK头文件、转码头文件
- └ lib : SDK动态库、第三方库、转码库
- └ module
 - └ GigEDriver : GigE相机驱动程序
 - └ USBDriver : USB相机的驱动程序
- └ share
 - └ C : C例程
 - └ Qt : Qt例程
 - └ Python : Python例程

1.3. SDK（Python 接口）对应的资源

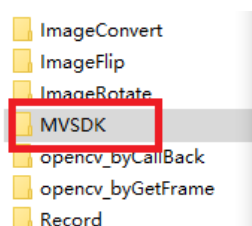
大类别	小类别	文件位置
SDK 库	头文件	【SDK 安装目录】 \Development\Samples\Python\IMV\IMVApi.py \Development\Samples\Python\IMV\IMVDefines.py
	运行库（32 位）	【SDK 安装目录】\Runtime\Win32\MVSDKmd.dll
	运行库（64 位）	【SDK 安装目录】\Runtime\x64\MVSDKmd.dll
Python 例程	\	【SDK 安装目录】\Development\Sample\Python\IMV

1.4. SDK 工程开发配置

以 x64 为例演示工程配置方法。

①拷贝头文件

【SDK 安装目录】\Development\Sample\Python\IMV



②导入头文件模块

```
sys.path.append("../MVSDK")  
from IMVApi import *
```

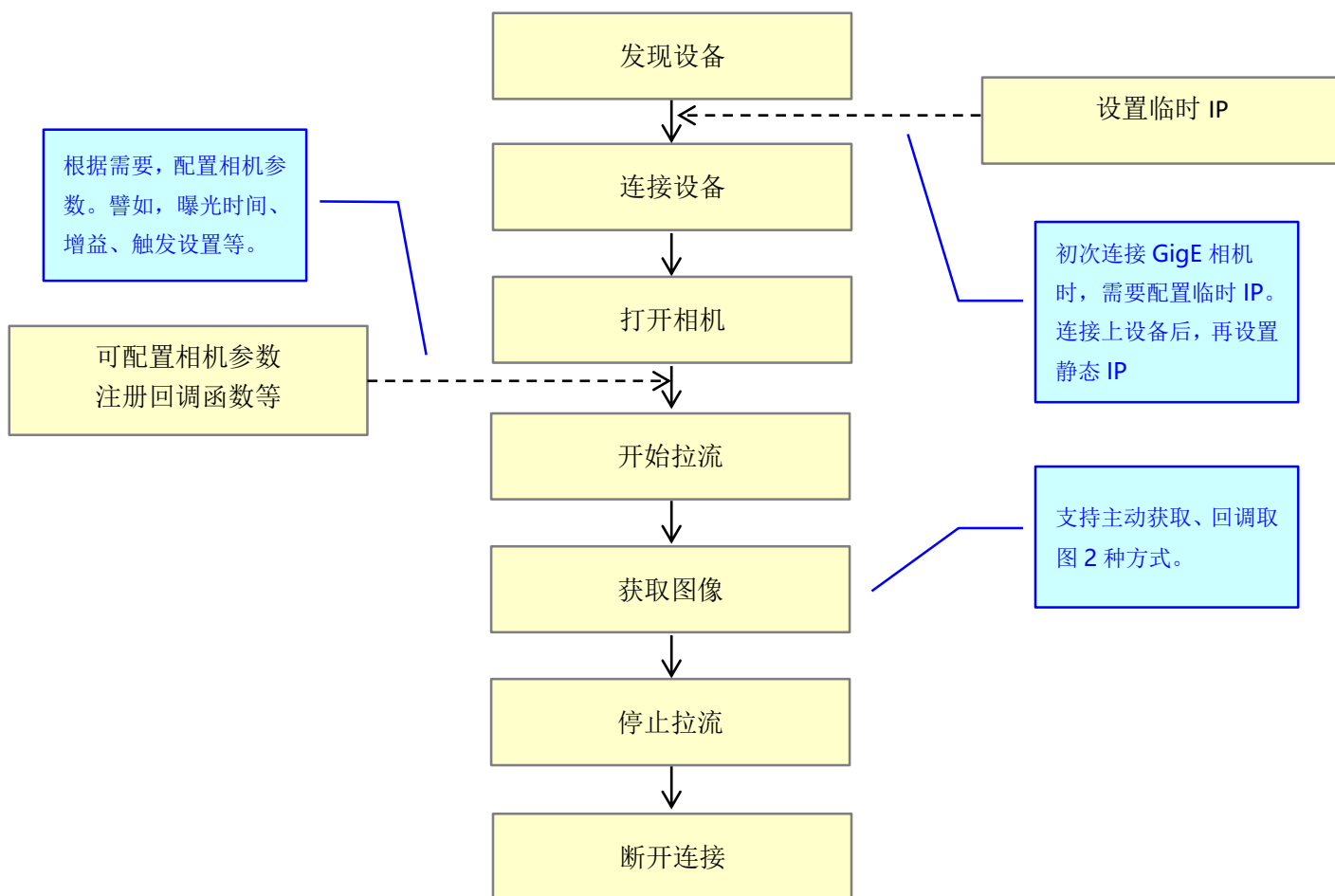
③导入动态库

可将头文件 IMVApi.py 中加载动态库的路径，修改为 MVSDKmd.dll 的绝对路径：

【SDK 安装目录】\Runtime\x64\MVSDKmd.dll

```
# 加载SDK动态库  
# load SDK library  
# 32bit  
#MVSDKdll = WinDLL("../../../../../Runtime/Win32/MVSDKmd.dll")  
# 64bit  
MVSDKdll = WinDLL("../../../../../Runtime/x64/MVSDKmd.dll")
```

2. SDK 的整体调用流程



3. SDK 的各功能接口调用说明

SDK 接口头文件路径：\Development\Samples\Python\IMV\MVSDK\IMVApi.py

SDK 接口的宏定义、枚举、结构体、回调函数声明路径：\Development\Samples\Python\IMV\MVSDK\IMVDefines.py

对象说明：self 指使用 MvCamera 类的实例对象

3.1. 系统操作

3.1.1. 获取版本信息

接口：IMV_GetVersion()

功能说明：获取版本信息

成功时返回版本信息，失败时返回NULL

参考例程：暂无

3.1.2. 枚举设备（发现设备）

接口：IMV_EnumDevices(pDeviceList, interfaceType)

参数说明：

参数	I/O	类型	说明
pDeviceList	OUT	IMV_DeviceList	设备列表
interfaceType	IN	c_uint	待枚举的接口类型，类型可任意组合，如 interfaceTypeGige interfaceTypeUsb3

功能说明：枚举设备

成功，返回IMV_OK，错误，返回错误码

- 1、当interfaceType = interfaceTypeAll 时，枚举所有接口下的在线设备
- 2、当interfaceType = interfaceTypeGige 时，枚举所有GigE网口下的在线设备
- 3、当interfaceType = interfaceTypeUsb3 时，枚举所有USB接口下的在线设备
- 4、当interfaceType = interfaceTypeCL 时，枚举所有CameraLink接口下的在线设备
- 5、该接口下的interfaceType支持任意接口类型的组合，如，若枚举所有GigE网口和USB3接口下的在线设备时，可将interfaceType设置为 interfaceType = interfaceTypeGige | interfaceTypeUsb3, 其它接口类型组合以此类推

参考例程：当前所有例程均有涉及

接口：IMV_EnumDevicesByUnicast(pDeviceList, pIpAddress)

参数说明：

参数	I/O	类型	说明
pDeviceList	OUT	IMV_DeviceList	设备列表
pIpAddress	IN	String	设备的IP地址

功能说明：以单播形式枚举设备，仅限 Gige 设备使用。

成功，返回IMV_OK；错误，返回错误码。
参考例程：暂无

3.1.3. 设备句柄的创建与销毁

接口：IMV_CreateHandle(self, mode, pIdentifier)

参数说明：

参数	I/O	类型	说明
mode	IN	IMV_ECreateHandleMode	创建设备方式
pIdentifier	IN	bytes	指定标示符(设备键、设备自定义名、IP地址为char类型指针强转void类型指针，索引为unsigned int类型指针强转void类型指针)

功能说明：通过指定标示符创建设备句柄，如指定索引、设备键、设备自定义名、IP地址。

成功，返回IMV_OK；错误，返回错误码。
参考例程：当前所有例程均有涉及

接口：IMV_DestroyHandle(self)

功能说明：销毁设备句柄

成功，返回IMV_OK；错误，返回错误码。
参考例程：当前所有例程均有涉及

3.2.相机操作

接口：IMV_GetDeviceInfo(self, pDevInfo)

参数说明：

参数	I/O	类型	说明
pDevInfo	OUT	IMV_DeviceInfo	设备信息

功能说明：获取设备信息

成功，返回IMV_OK；错误，返回错误码
参考例程：当前所有例程均有涉及

3.2.1. 相机的连接与关闭

接口：IMV_Open(self)

功能说明：打开设备（根据句柄连接设备）

成功，返回IMV_OK；错误，返回错误码
参考例程：当前所有例程均有涉及

接口：IMV_OpenEx(self, accessPermission)

参数说明：

参数	I/O	类型	说明
accessPermission	IN	IMV_ECameraAccessPermission	控制通道权限(IMV_Open默认使用accessPermissionControl权限)

功能说明：打开设备（根据句柄与访问权限连接设备）

成功，返回IMV_OK；错误，返回错误码

与IMV_Open接口的区别在于连接操作执行时需判断访问权限

参考例程：暂无

接口： `IMV_IsOpen(self)`

功能说明：判断设备是否已打开

打开状态，返回true；关闭状态或者掉线状态，返回false

参考例程：暂无

接口： `IMV_Close(self)`

功能说明：关闭设备（根据句柄断开设备）

成功，返回IMV_OK；错误，返回错误码

参考例程：当前所有例程均有涉及

3.2.2. 相机配置的下载、保存及加载配置

接口： `IMV_DownloadGenICamXML(self, pFullFileName)`

参数说明：

参数	I/O	类型	说明
pFullFileName	IN	str	文件要保存的路径

功能说明：下载设备描述XML文件，并保存到指定路径，如：D:\\xml.zip

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_SaveDeviceCfg(self, pFullFileName)`

参数说明：

参数	I/O	类型	说明
pFullFileName	IN	str	导出的设备配置文件全名(含路径)，如：D:\\config.xml 或 D:\\config.mvcfg

功能说明：保存设备配置到指定的位置。同名文件已存在时，覆盖。

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_LoadDeviceCfg(self, pFullFileName, pErrorList)`

参数说明：

参数	I/O	类型	说明
pFullFileName	IN	str	导出的设备配置文件全名(含路径)如：D:\\config.xml或D:\\config.mvcfg

pErrorList	OUT	IMV_ErrorList	加载失败的属性名列表。存放加载失败的属性上限为 IMV_MAX_ERROR_LIST_NUM
------------	-----	---------------	---

功能说明：从文件加载设备xml配置

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

3.2.3. 相机配置设置（GigE 相机特有）

接口： `IMV_GIGE_ForceIpAddress(self, pIpAddress, pSubnetMask, pGateway)`

参数说明：

参数	I/O	类型	说明
pIpAddress	IN	str	导出的设备配置文件全名(含路径)， 如：D:\\config.xml 或 D:\\config.mvcfg
pSubnetMask	IN	str	加载失败的属性名列表。存放加载失败的属性上限为 IMV_MAX_ERROR_LIST_NUM
pGateway	IN	str	默认网关

功能说明：修改设备IP，仅限Gige设备使用

成功，返回IMV_OK；错误，返回错误码

- 1、调用该函数时如果pSubnetMask和pGateway都设置了有效值，则以此有效值为准；
- 2、调用该函数时如果pSubnetMask和pGateway都设置了NULL，则内部实现时用它所连接网卡的子网掩码和网关代替
- 3、调用该函数时如果pSubnetMask和pGateway两者中其中一个为NULL，另一个非NULL，则返回错误

参考例程：暂无

接口： `IMV_GIGE_GetAccessPermission(self, pAccessPermission)`

参数说明：

参数	I/O	类型	说明
pAccessPermission	OUT	IMV_ECameraAccessPermission	设备的当前访问权限

功能说明：获取设备的当前访问权限，仅限Gige设备使用

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_GIGE_SetAnswerTimeout(self, timeout)`

参数说明：

参数	I/O	类型	说明
timeout	IN	c_uint	超时时间，单位ms

功能说明：设置设备对sdk命令的响应超时时间, 仅限Gige设备使用

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

3.2.4. 本品牌相机特有功能

接口： `IMV_WriteUserPrivateData(self, pBuffer, pLength)`

参数说明：

参数	I/O	类型	说明
pBuffer	IN	array	数据缓冲的指针
pLength	IN	c_int	期望写入的字节数 [OUT] 实际写入的字节数

功能说明：写用户自定义数据。相机内部保留32768字节用于用户存储自定义数据(此功能针对本品牌相机，其它品牌相机无此功能)

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_ReadUserPrivateData(self, pBuffer, pLength)`

参数说明：

参数	I/O	类型	说明
pBuffer	OUT	array	数据缓冲的指针
pLength	IN	c_int	期望写入的字节数 [OUT] 实际写入的字节数

功能说明：读用户自定义数据。相机内部保留32768字节用于用户存储自定义数据(此功能针对本品牌相机，其它品牌相机无此功能)

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_WriteUARTData(self, pBuffer, pLength)`

参数说明：

参数	I/O	类型	说明
pBuffer	IN	array	数据缓冲的指针
pLength	IN	c_int	期望写入的字节数 [OUT] 实际写入的字节数

功能说明：往相机串口寄存器写数据，每次写会清除掉上次的数据(此功能只支持包含串口功能的本品牌相机)

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_ReadUARTData(self, pBuffer, pLength)`

参数说明：

参数	I/O	类型	说明
pBuffer	OUT	array	数据缓冲的指针
pLength	IN	c_int	期望写入的字节数 [OUT] 实际写入的字节数

功能说明：从相机串口寄存器读取串口数据(此功能只支持包含串口功能的本品牌相机)

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

3.3.注册事件回调

接口： `IMV_SubscribeConnectArg(self, proc, pUser)`

参数说明：

参数	I/O	类型	说明
proc	IN	IMV_ConnectCallBack	设备连接状态事件回调函数
pUser	IN	c_void_p	用户自定义数据，可设为None

功能说明：设备连接状态事件回调注册

成功，返回IMV_OK；错误，返回错误码

只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册

参考例程：暂无

接口： `IMV_SubscribeParamUpdateArg(self, proc, pUser)`

参数说明：

参数	I/O	类型	说明
proc	IN	IMV_ParamUpdateCallBack	参数更新注册的事件回调函数
pUser	IN	c_void_p	用户自定义数据，可设为None

功能说明：参数更新事件回调注册

成功，返回IMV_OK；错误，返回错误码

只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册

参考例程：暂无

接口： `IMV_SubscribeStreamArg(self, proc, pUser)`

参数说明：

参数	I/O	类型	说明
proc	IN	IMV_StreamCallBack	流通道事件回调注册函数
pUser	IN	c_void_p	用户自定义数据，可设为None

功能说明：流通道事件回调注册

成功，返回IMV_OK；错误，返回错误码

只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册

参考例程：暂无

接口： `IMV_SubscribeMsgChannelArg(self, proc, pUser)`

参数说明：

参数	I/O	类型	说明
proc	IN	IMV_MsgChannelCallBack	消息通道事件回调注册函数
pUser	IN	c_void_p	用户自定义数据，可设为None

功能说明：消息通道事件回调注册

成功，返回IMV_OK；错误，返回错误码

只支持一个回调函数,且设备关闭后，注册会失效，打开设备后需重新注册

参考例程：暂无

3.4.相机流资源操作

3.4.1. 流数据设置

接口： `IMV_SetBufferCount(self, nSize)`

参数说明：

参数	I/O	类型	说明
nSize	IN	int	缓存数量

功能说明：设置帧数据缓存个数

成功，返回IMV_OK；错误，返回错误码

不能在拉流过程中设置

参考例程：暂无

接口： `IMV_ClearFrameBuffer(self)`

功能说明：清除帧数据缓存

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_GIGE_SetInterPacketTimeout(self, nTimeout)`

参数说明：

参数	I/O	类型	说明
nTimeout	IN	int	包间隔时间，单位是毫秒

功能说明：设置驱动包间隔时间(MS), 仅对Gige设备有效

成功，返回IMV_OK；错误，返回错误码

触发模式尾包丢失重传机制

参考例程：暂无

接口： `IMV_GIGE_SetSingleResendMaxPacketNum(self, maxPacketNum)`

参数说明：

参数	I/O	类型	说明
maxPacketNum	IN	int	单次重传最大包个数

功能说明：设置单次重传最大包个数，仅对GigE设备有效

成功，返回IMV_OK；错误，返回错误码

maxPacketNum为0时，该功能无效

参考例程：暂无

接口： `IMV_GIGE_SetMaxLostPacketNum(self, maxLostPacketNum)`

参数说明：

参数	I/O	类型	说明
maxLostPacketNum	IN	int	最大丢包的数量

功能说明：设置同一帧最大丢包的数量, 仅对GigE设备有效

成功，返回IMV_OK；错误，返回错误码

maxLostPacketNum为0时，该功能无效

只支持一个回调函数,且设备关闭后,注册会失效,打开设备后需重新注册

参考例程: 暂无

接口: `IMV_USB_SetUrbTransfer(self, nNum, nSize)`

参数说明:

参数	I/O	类型	说明
nNum	IN	int	传输数据块的数量(范围:5-256)
nSize	IN	int	传输数据块的大小(范围:8-512,单位:KByte)

功能说明: 设置U3V设备的传输数据块的数量和大小,仅对USB设备有效

成功,返回IMV_OK; 错误,返回错误码

1、该接口暂时只有在Linux平台且使用无驱的情况下设置才有效,其他情况下返回

IMV_NOT_SUPPORT错误码

2、传输数据块数量,范围5 - 256,默认为64,该值设置过小会影响传输效率;多台相机共同使用时,可以适当减小该值

3、传输每个数据块大小,范围8 - 512,默认为64,单位是KByte

参考例程: 暂无

3.4.2. 图像采集

接口: `IMV_StartGrabbing(self)`

功能说明: 开始取流

成功,返回IMV_OK; 错误,返回错误码

参考例程: Grab

接口: `IMV_StartGrabbingEx(self, maxImagesGrabbed, strategy)`

参数说明:

参数	I/O	类型	说明
maxImagesGrabbed	IN	int	允许最多的取帧数,达到指定取帧数后停止取流,如果为0,表示忽略此参数连续取流 (IMV_StartGrabbing默认0)
strategy	IN	IMV_EGrabStrategy	取流策略,(IMV_StartGrabbing默认使用 grabStrategySequential取流)

功能说明: 开始取流

成功,返回IMV_OK; 错误,返回错误码

参考例程: 暂无

接口: `IMV_IsGrabbing(self)`

功能说明: 判断设备是否正在取流

正在取流,返回true; 不在取流,返回false

参考例程: 暂无

接口： `IMV_StopGrabbing(self)`

功能说明：停止取流

成功，返回IMV_OK；错误，返回错误码

参考例程：Grab

3.4.3. 获取图像数据

接口： `IMV_AttachGrabbing(self, proc, pUser)`

参数说明：

参数	I/O	类型	说明
proc	IN	IMV_FrameCallBack	帧数据信息回调函数，建议不要在该函数中处理耗时的操作，否则会阻塞后续帧数据的实时性
pUser	IN	c_void_p	用户自定义数据，可设为None

功能说明：注册帧数据回调函数(异步获取帧数据机制)

成功，返回IMV_OK；错误，返回错误码

该异步获取帧数据机制和同步获取帧数据机制(IMV_GetFrame)互斥，对于同一设备，系统中两者只能选其一

只支持一个回调函数，且设备关闭后，注册会失效，打开设备后需重新注册

参考例程：ChunkData、GrabCallback

接口： `IMV_GetFrame(self, pFrame, timeoutMS)`

参数说明：

参数	I/O	类型	说明
pFrame	OUT	IMV_Frame	帧数据信息
timeoutMS	IN	int	获取一帧图像的超时时间，INFINITE时表示无限等待，直到收到一帧数据或者停止取流。单位是毫秒

功能说明：获取一帧图像(同步获取帧数据机制)

成功，返回IMV_OK；错误，返回错误码

该接口不支持多线程调用。

该同步获取帧机制和异步获取帧机制(IMV_AttachGrabbing)互斥，对于同一设备，系统中两者只能选其一。

使用内部缓存获取图像，需要IMV_ReleaseFrame进行释放图像缓存。

参考例程：Grab

接口： `IMV_ReleaseFrame(self, pFrame)`

参数说明：

参数	I/O	类型	说明
pFrame	IN	IMV_Frame	帧数据信息

功能说明：释放图像缓存

成功，返回IMV_OK；错误，返回错误码

参考例程：Grab

3.4.4. 流与图像数据的其他操作

接口： `IMV_CloneFrame(self, pFrame, pCloneFrame)`

参数说明：

参数	I/O	类型	说明
pFrame	IN	IMV_Frame	克隆源帧数据信息
pCloneFrame	OUT	IMV_Frame	新的帧数据信息

功能说明：帧数据深拷贝克隆

成功，返回IMV_OK；错误，返回错误码

使用IMV_ReleaseFrame进行释放图像缓存。

参考例程：暂无

接口： `IMV_GetChunkDataByIndex(self, pFrame, index, pChunkDataInfo)`

参数说明：

参数	I/O	类型	说明
pFrame	IN	IMV_Frame	帧数据信息
index	IN	int	索引ID
pChunkDataInfo	OUT	IMV_ChunkDataInfo	Chunk数据信息

功能说明：获取Chunk数据(仅对GigE/Usb相机有效)

成功，返回IMV_OK；错误，返回错误码

参考例程：ChunkData

接口： `IMV_GetStatisticsInfo(self, pStreamStatsInfo)`

参数说明：

参数	I/O	类型	说明
pStreamStatsInfo	OUT	IMV_StreamStatisticsInfo	流统计信息数据

功能说明：获取流统计信息(IMV_StartGrabbing / IMV_StartGrabbing执行后调用)

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_ResetStatisticsInfo(self)`

功能说明：重置流统计信息(IMV_StartGrabbing / IMV_StartGrabbing执行后调用)

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

3.5.属性操作

3.5.1. 属性的判断、获取、设置

① 属性的判断

接口： `IMV_FeatureIsAvailable(self, pFeatureName)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

功能说明：判断属性是否可用

可，返回true；不可，返回false

参考例程：暂无

接口： `IMV_FeatureIsReadable(self, pFeatureName)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

功能说明：判断属性是否可读

可，返回true；不可，返回false

参考例程：暂无

接口： `IMV_FeatureIsWriteable(self, pFeatureName)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

功能说明：判断属性是否可写

可，返回true；不可，返回false

参考例程：暂无

接口： `IMV_FeatureIsStreamable(self, pFeatureName)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

功能说明：判断属性是否可流

可，返回true；不可，返回false

参考例程：暂无

接口： `IMV_FeatureIsValid(self, pFeatureName)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

功能说明：判断属性是否有效

可，返回true；不可，返回false

参考例程：Record

② 属性的获取

接口： `IMV_GetIntFeatureValue(self, pFeatureName, pIntValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

pIntValue	OUT	c_int	整型属性值
-----------	-----	-------	-------

功能说明：获取整型属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetIntFeatureMin(self, pFeatureName, pIntValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pIntValue	OUT	c_int	整型属性可设的最小值

功能说明：获取整型属性可设的最小值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetIntFeatureMax(self, pFeatureName, pIntValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pIntValue	OUT	c_int	整型属性可设的最大值

功能说明：获取整型属性可设的最大值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetIntFeatureInc(self, pFeatureName, pIntValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pIntValue	OUT	c_int	整型属性步长

功能说明：获取整型属性步长

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetDoubleFeatureValue(self, pFeatureName, pDoubleValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pDoubleValue	OUT	c_double	浮点属性值

功能说明：获取浮点属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetDoubleFeatureMin(self, pFeatureName, pDoubleValue)`

参数说明：

参数	I/O	类型	说明
----	-----	----	----

pFeatureName	IN	str	属性名
pDoubleValue	OUT	c_double	浮点属性可设的最小值

功能说明：获取浮点属性可设的最小值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetDoubleFeatureMax(self, pFeatureName, pDoubleValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pDoubleValue	OUT	c_double	浮点属性可设的最大值

功能说明：获取浮点属性可设的最大值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetBoolFeatureValue(self, pFeatureName, pBoolValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pBoolValue	OUT	c_bool	布尔属性值

功能说明：获取布尔属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetStringFeatureValue(self, pFeatureName, pStringValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pStringValue	OUT	IMV_String	字符串属性值

功能说明：获取字符串属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口：`IMV_GetEnumFeatureValue(self, pFeatureName, pEnumValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pEnumValue	OUT	c_int	枚举属性值

功能说明：获取枚举属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：CommPropAccess

接口：`IMV_GetEnumFeatureSymbol(self, pFeatureName, pEnumSymbol)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pEnumSymbol	OUT	IMV_String	枚举属性symbol值

功能说明：获取枚举属性symbol值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_GetEnumFeatureEntryNum(self, pFeatureName, pEntryNum)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pEntryNum	OUT	c_int	枚举属性值可设置个数

功能说明：获取枚举属性值可设置个数

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_GetEnumFeatureEntrys(self, pFeatureName, pEnumEntryList)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pEnumEntryList	OUT	IMV_EnumEntryList	枚举属性值可设置枚举值列表

功能说明：获取枚举属性值可设置枚举值列表

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

③ 属性的设置

接口： `IMV_SetIntFeatureValue(self, pFeatureName, pIntValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pIntValue	IN	int	待设置的整型属性值

功能说明：设置整型属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_SetDoubleFeatureValue(self, pFeatureName, doubleValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
doubleValue	IN	double	待设置的浮点属性值

功能说明：设置浮点属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_SetBoolFeatureValue(self, pFeatureName, boolValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
boolValue	IN	bool	待设置的布尔属性值

功能说明：设置布尔属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：ChunkData

接口： `IMV_SetEnumFeatureValue(self, pFeatureName, enumValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
enumValue	IN	int	待设置的枚举属性值

功能说明：设置枚举属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_SetEnumFeatureSymbol(self, pFeatureName, pEnumSymbol)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pEnumSymbol	IN	str	待设置的枚举属性symbol值

功能说明：设置枚举属性symbol值

成功，返回IMV_OK；错误，返回错误码

参考例程：ChunkData

接口： `IMV_SetStringFeatureValue(self, pFeatureName, pStringValue)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名
pStringValue	IN	str	待设置的字符串属性值

功能说明：设置字符串属性值

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

3.5.2. 命令属性的执行

接口： `IMV_ExecuteCommandFeature(self, pFeatureName)`

参数说明：

参数	I/O	类型	说明
pFeatureName	IN	str	属性名

功能说明： 执行命令属性
成功，返回IMV_OK；错误，返回错误码
参考例程： ChunkData

3.5.3. 属性的查询与使用

打开相机客户端可查询对应的属性，并且可以了解属性的详细信息，如：数据类型、范围等。

1) 整型属性

例：若需获取相机的像素高度信息，找到 **Height** 属性并选中，下方会显示该属性的数据类型，根据 **Type** 的显示而定，下图 **Height** 属性的数据类型为 **int** 整型。



随后对所找到属性进行获取及修改。

```
heightValue = c_int64(0)
heightMinValue = c_int64(0)
heightMaxValue = c_int64(0)
incrementValue = c_int64(0)

// 获取属性值
ret = cam.IMV_GetIntFeatureValue("Height", heightValue)
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 获取属性可设的最小值，若无最小值则此操作错误
ret = cam.IMV_GetIntFeatureMin("Height", heightMinValue)
```



```

if (IMV_OK != ret):
    print("Get feature minimum value failed! ErrorCode[%d] "% ret)
    return ret

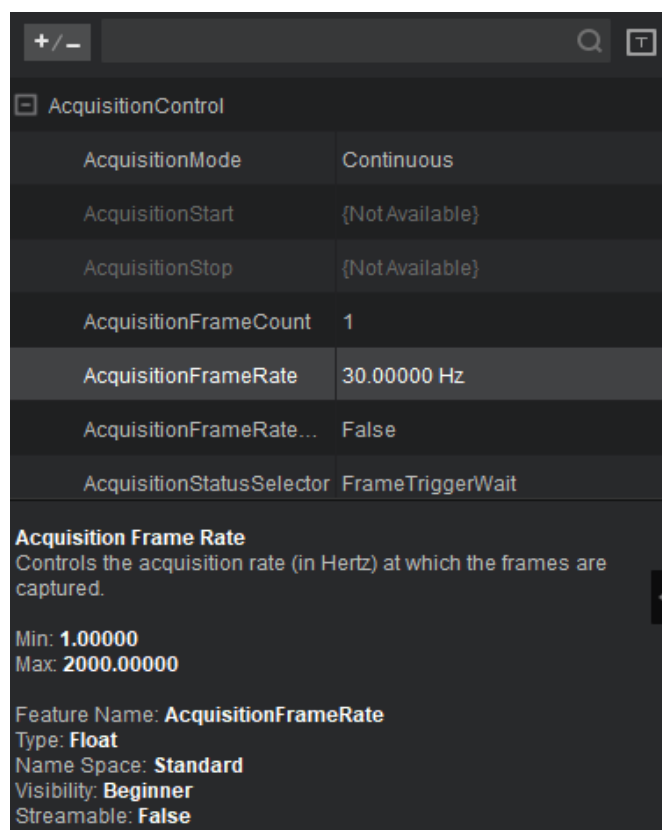
// 获取属性可设的最大值，若无最大值则此操作错误
ret = cam.IMV_GetIntFeatureMax("Height", heightMaxValue)
if (IMV_OK != ret):
    print("Get feature maximum value failed! ErrorCode[%d]"% ret)
    return ret

// 获取属性步长，若无步长值则此操作错误
ret = cam.IMV_GetIntFeatureInc("Height", incrementValue)
if (IMV_OK != ret):
    print("Get feature increment value failed! ErrorCode[%d]"% ret)
    return ret

// 设置属性值
ret = cam.IMV_SetIntFeatureValue("Height", heightValue.value)
if (IMV_OK != ret):
    print("Set feature value failed! ErrorCode[%d]"% ret)
    return ret

```

2) 浮点型属性



```

framerate = c_ double(0)
frameratemin = c_ double(0)

```

```
framerate_max = c_double(0)

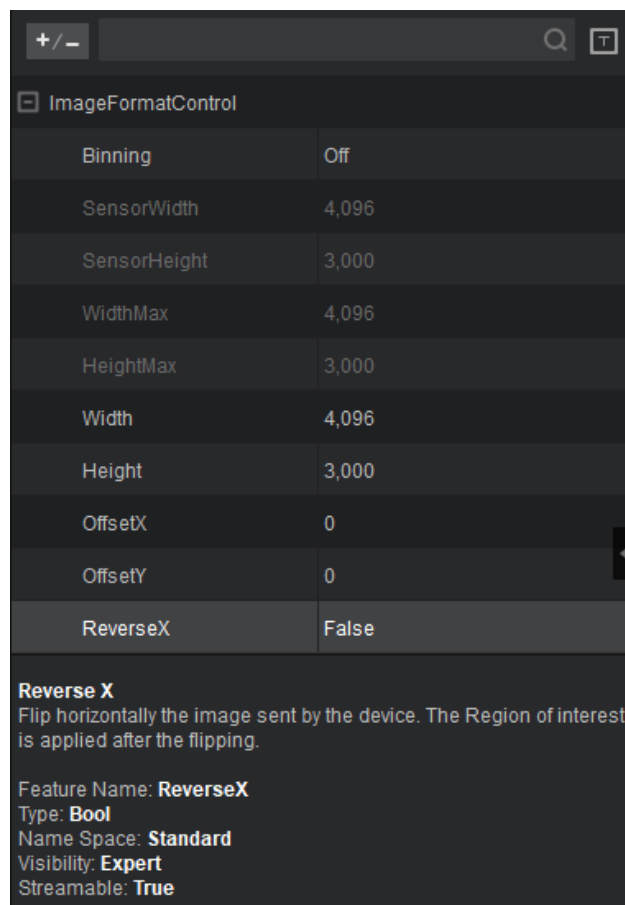
// 获取属性值
ret = cam.IMV_GetDoubleFeatureValue("AcquisitionFrameRate", framerate)
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 获取属性可设的最小值，若无最小值则此操作错误
ret = cam.IMV_GetDoubleFeatureMin("AcquisitionFrameRate", framerate_min)
if (IMV_OK != ret):
    print("Get feature minimum value failed! ErrorCode[%d]" % ret)
    return ret

// 获取属性可设的最大值，若无最大值则此操作错误
ret = cam.IMV_GetDoubleFeatureMax("AcquisitionFrameRate", framerate_max)
if (IMV_OK != ret):
    print("Get feature maximum value failed! ErrorCode[%d]" % ret)
    return ret

// 设置属性值
ret = cam.IMV_SetDoubleFeatureValue("AcquisitionFrameRate", framerate.value)
if (IMV_OK != ret):
    print("Set feature value failed! ErrorCode[%d]" % ret)
    return ret
```

3) 布尔属性



```
reversevalue = c_bool(0)
```

```
// 获取属性值
```

```
ret = cam.IMV_GetBoolFeatureValue ("ReverseX", reversevalue)
```

```
if (IMV_OK != ret):
```

```
    print("Get feature value failed! ErrorCode[%d]" % ret)
```

```
    return ret
```

```
// 设置属性值
```

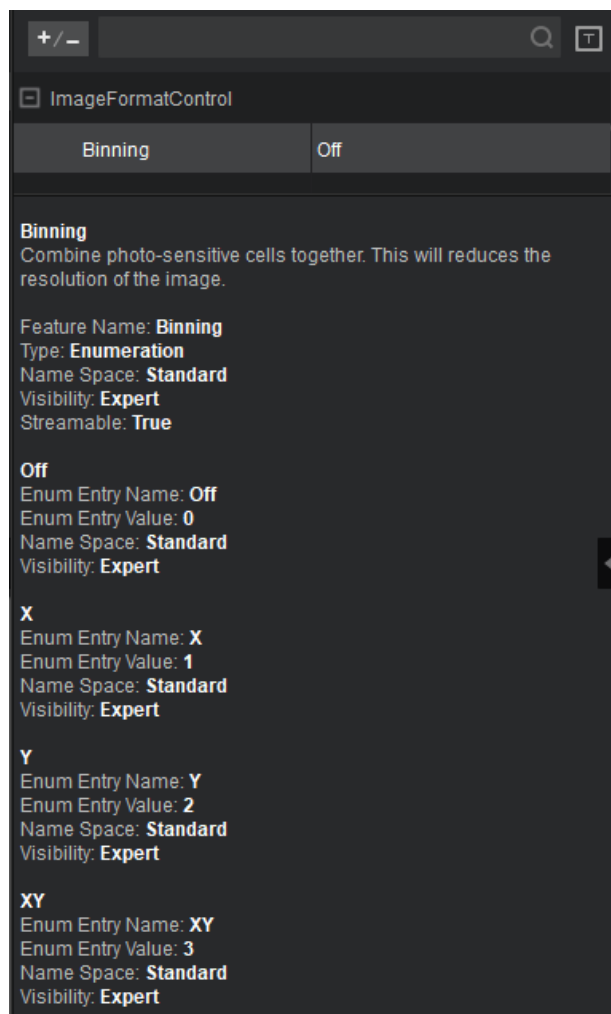
```
ret = cam.IMV_SetBoolFeatureValue("ReverseX", reversevalue.value)
```

```
if (IMV_OK != ret):
```

```
    print("Set feature value failed! ErrorCode[%d]", ret)
```

```
    return ret
```

4) 枚举属性



```
enumValue = c_uint64(0)
pEnumSymbol = IMV_String()
pEntryNum = c_uint(0)
pEnumEntryList = IMV_EnumEntryList()

// 获取属性值的可设枚举值列表
ret = cam.IMV_GetEnumFeatureValue("Binning", pEnumEntryList)
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 获取属性可设枚举值的个数
ret = cam.IMV_GetEnumFeatureEntryNum("Binning", pEntryNum)
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 获取属性symbol值
ret = cam.IMV_GetEnumFeatureSymbol("Binning", pEnumSymbol)
```

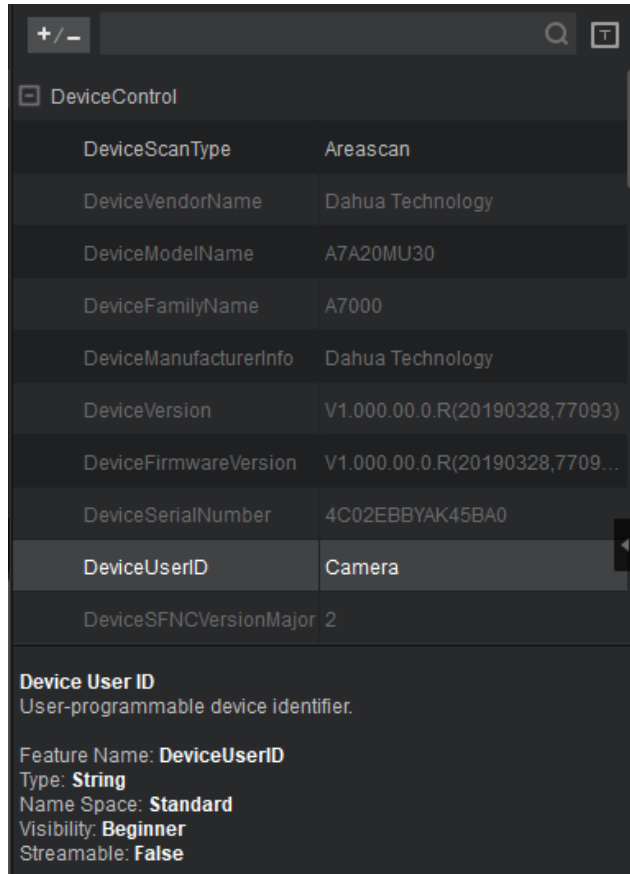
```
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 获取属性值
ret = cam.IMV_GetIntFeatureInc("Binning", enumValue)
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 设置属性symbol值
ret = cam.IMV_SetEnumFeatureValue("Binning", pEnumSymbol)
if (IMV_OK != ret):
    print("Set feature value failed! ErrorCode[%d]" % ret)
    return ret

// 设置属性值
ret = cam.IMV_SetEnumFeatureValue("Binning", enumValue.value)
if (IMV_OK != ret):
    print("Set feature value failed! ErrorCode[%d]" % ret)
    return ret
```

5) 字符串属性



DeviceScanType	Areascan
DeviceVendorName	Dahua Technology
DeviceModelName	A7A20MU30
DeviceFamilyName	A7000
DeviceManufacturerInfo	Dahua Technology
DeviceVersion	V1.000.00.0.R(20190328,77093)
DeviceFirmwareVersion	V1.000.00.0.R(20190328,7709...
DeviceSerialNumber	4C02EBBYAK45BA0
DeviceUserID	Camera
DeviceSFNCVersionMajor	2

Device User ID
User-programmable device identifier.

Feature Name: **DeviceUserID**
Type: **String**
Name Space: **Standard**
Visibility: **Beginner**
Streamable: **False**

```

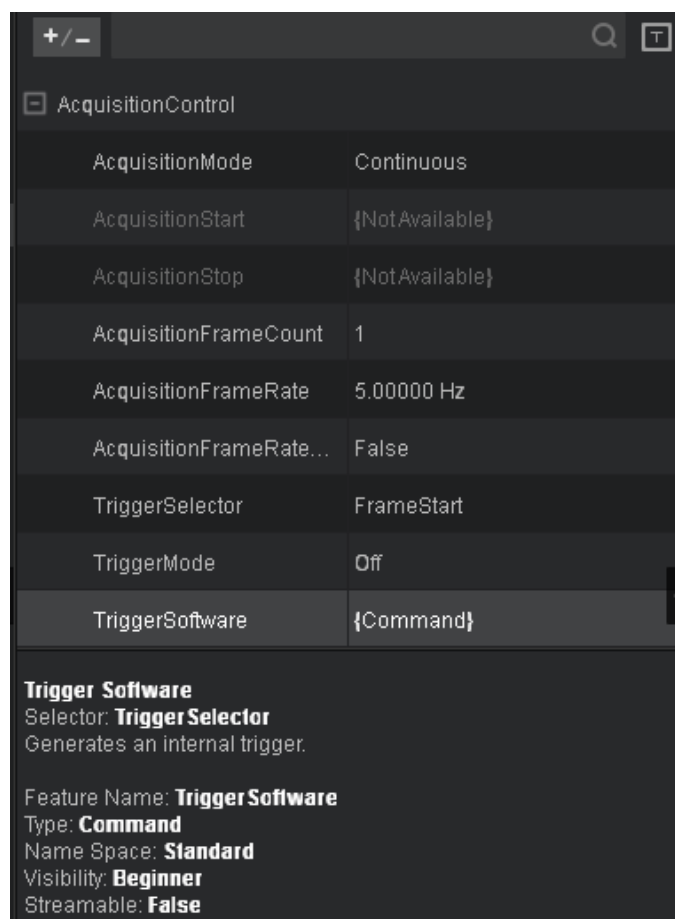
id = IMV_String()

// 获取属性值
ret = cam.IMV_GetStringFeatureValue ("DeviceUserID", id)
if (IMV_OK != ret):
    print("Get feature value failed! ErrorCode[%d]" % ret)
    return ret

// 设置属性值
ret = cam.IMV_SetStringFeatureValue("DeviceUserID", id)
if (IMV_OK != ret):
    print("Set feature value failed! ErrorCode[%d]" % ret)
    return ret

```

6) 命令属性



```

// 执行命令属性值
ret = cam.IMV_ExecuteCommandFeature ("TriggerSoftware")
if (IMV_OK != ret):
    print("Execute Command feature failed! ErrorCode[%d] " % ret)
    return ret

```

3.6. 图像操作

3.6.1. 录像相关

接口： `IMV_OpenRecord(self, pstRecordParam)`

参数说明：

参数	I/O	类型	说明
pstRecordParam	IN	IMV_RecordParam	录像参数结构体

功能说明：打开录像

成功，返回IMV_OK；错误，返回错误码

参考例程：Record

接口： `IMV_InputOneFrame(self, pstRecordFrameInfoParam)`

参数说明：

参数	I/O	类型	说明
pstRecordFrameInfoParam	IN	IMV_RecordFrameInfoParam	录像用帧信息结构体

功能说明：录制一帧图像

成功，返回IMV_OK；错误，返回错误码

参考例程：暂无

接口： `IMV_CloseRecord(self)`

功能说明：关闭录像

成功，返回IMV_OK；错误，返回错误码

参考例程：Record

3.6.2. 图像相关

接口： `IMV_PixelConvert(self, pstPixelConvertParam)`

参数说明：

参数	I/O	类型	说明
pstPixelConvertParam	IN/OUT	IMV_PixelConvertParam	像素格式转换参数结构体

功能说明：像素格式转换

成功，返回IMV_OK；错误，返回错误码

只支持转化成目标像素格式gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8 / gvspPixelBGRA8

通过该接口将原始图像数据转换成用户所需的像素格式并存放在调用者指定内存中。

像素格式为YUV411Packed的时，图像宽须能被4整除

像素格式为YUV422Packed的时，图像宽须能被2整除

像素格式为YUYVPacked的时，图像宽须能被2整除

转换后的图像:数据存储是从最上面第一行开始的，这个是相机数据的默认存储方向

参考例程：ImageConvert

接口： `IMV_FlipImage(self, pstFlipImageParam)`

参数说明：

参数	I/O	类型	说明
pstFlipImageParam	IN/OUT	IMV_FlipImageParam	图像翻转参数结构体

功能说明：图像翻转

成功，返回IMV_OK；错误，返回错误码

只支持像素格式gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8的图像的垂直和水平翻转。

通过该接口将原始图像数据翻转后并存放在调用者指定内存中。

参考例程：ImageFlip

接口： `IMV_RotateImage(self, pstRotateImageParam)`

参数说明：

参数	I/O	类型	说明
pstRotateImageParam	IN/OUT	IMV_RotateImageParam	图像旋转参数结构体

功能说明：图像顺时针旋转

成功，返回IMV_OK；错误，返回错误码

只支持gvspPixelRGB8 / gvspPixelBGR8 / gvspPixelMono8格式数据的90/180/270度顺时针旋转。

通过该接口将原始图像数据旋转后并存放在调用者指定内存中。

参考例程：ImageRotate

接口： `IMV_SaveImageToFile(self, pstSaveFileParam)`

参数说明：

参数	I/O	类型	说明
pstSaveFileParam	IN	IMV_SaveImageToFileParam	保存图片文件参数结构体

功能说明：图像顺时针旋转

成功，返回IMV_OK；错误，返回错误码

该接口支持BMP/JPEG/PNG/TIFF

JPEG格式最大支持宽高为65500

参考例程：SaveImageToFile

4. 配套例程说明

Python 例程路径：【SDK 安装目录】\Development\Samples\Python\IMV

例程名	例程说明
ChunkData	获取块数据
Grab	自由模式采图（主动取流）
GrabCallback	拉流回调
ImageConvert	图像转化
ImageFlip	图像翻转
ImageRotate	图像顺时针旋转
opencv_byCallBack	回调拉流使用 cv2 显示
opencv_byGetFrame	主动取流使用 cv2 显示
MultiCameraGrab	多相机使用例程
SaveImageToFile	保存图片例程
SoftTrigger	软件触发方式取流

5. 常见问题&注意点

5.1. 使用 python 接口采图，只能取到 8 张图像（和 SDK 缓存个数一致）。

原因： python 接口取到图像对象后，没有释放，导致缓存被占满，无法再取到流。

解决方法：如下图所示，每取到帧 **frame** 都需要调用 `IMV_ReleaseFrame` 接口进行释放。

```
#获取一帧图像
#get a frame image
nRet=cam.IMV_GetFrame(frame,500)
if IMV_OK!=nRet:
    print("Get frame failed!ErrorCode[%d]" % nRet)
    continue
print("Get frame blockId = [%d]"% frame.frameInfo.blockId)

#释放图像缓存
#release frame buffer
nRet=cam.IMV_ReleaseFrame(frame)
if IMV_OK!=nRet:
    print("Release frame failed! ErrorCode[%d]" % nRet)
```

- END -